

Vibe Coding을 위한 개발도구



1. 도구 소개

❖IDE 기반 도구

- AI 전용 IDE로 제작된 개발 도구
- 대표적인 예
 - VSCode + AI 관련 Extension(무료)
 - 사용 가능한 AI 기능 : Github Copilot, Claude, Gemini 등
 - Cursor, Windsurf, Antigravity : 유료 구독 방식
 - VSCode 기반 : VSCode의 코어 에디터와 UI를 활용
 - » VSCode의 Extension을 모두 지원하는 것은 아님

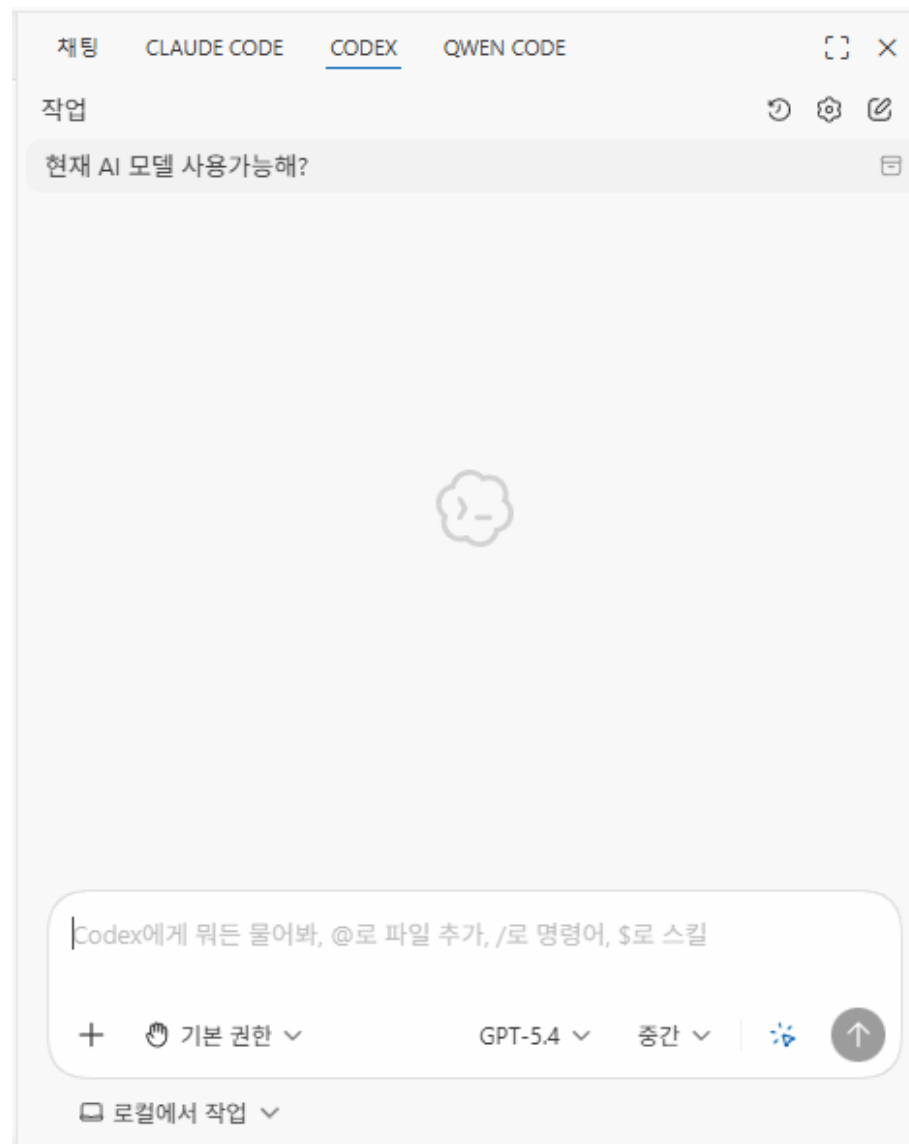
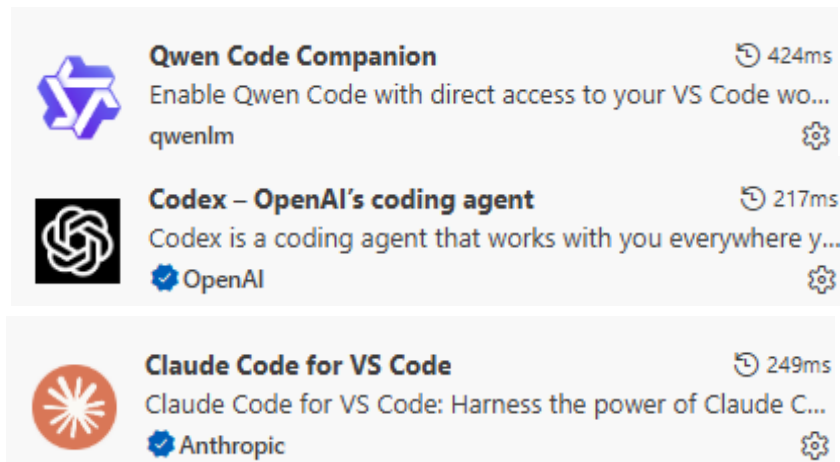
❖CLI 기반 도구

- 터미널에서 명령어를 이용하도록 만들어진 개발 도구
 - 터미널에서 실행하므로 어느 도구에나 연동이 가능함
- 대표적인 예
 - Claude Code : Claude
 - Antigravity CLI : Google Gemini를 이용한 CLI 도구
 - Codex : Open AI Chat GPT
 - OpenCode : 여러 AI 모델을 연동할 수 있는 CLI 개발 도구

2. VSCode 기반 AI 코딩 도구

❖ VSCode 확장 프로그램 기반의 도구

- Codex
- Claude Code
- Antigravity - 별도의 IDE 도구 사용
- Qwen
- Cursor



3. Claude Code 기본

❖ Claude Code란?

- Anthropic이 만든 Claude 모델을 기반으로 한 터미널 기반 AI 코딩 어시스턴트
 - 터미널 기반이므로 특정 IDE 환경에 종속되지 않음
- 단순 코드 자동 완성이 아닌 대화형 코딩에 좀더 초점을 맞추고 있음
 - AI Pair Programmer
- 이용 제한
 - Claude Pro 이상의 구독을 한 사용자
 - 비용을 지불하고 API Key를 발급받은 사용자
- SubAgent, Command, Skills, Workflow와 같은 강력한 기능 제공
- IDE 확장 지원
 - VSCode, Cursor, PyCharm, Android Studio 등 다양한 IDE 도구를 연동할 수 있음
 - 주요 기능
 - 자동 설치 : IDE에 내장된 터미널에서 Claude Code를 실행하면 자동으로 Extension 설치
 - 선택된 문자열 컨텍스트 추가 : 편집기에서 선택한 텍스트가 자동으로 Claude 컨텍스트에 추가됨
 - Diff 보기 : Claude Code에 의해 변경된 코드를 변경전과 비교하도록 화면 제공
 - 문자열 전송 : 선택된 문자열을 CTRL+ALT+K(맥은 CMD+ALT+K) 단축키로 프롬프트에 전송할 수 있음

3. Claude Code

❖ Claude Code 실행 명령어

- `claude [options] [command] [prompt]`

❖ options

- `--dangerously-skip-permissions`
 - 권한 체크를 skip함. 인터넷에 접속할 수 없는 샌드박스 환경에서만 사용할 것
- `--append-system-prompt <prompt>`
 - 입력한 프롬프트를 시스템 프롬프트에 추가함
 - 시스템 프롬프트는 사용자의 컨텍스트에 가장 높은 우선순위로 포함됨
- `--permission-mode <mode>`
 - 이번 세션을 위한 권한 모드 설정. "acceptEdits", "bypassPermissions", "default", "plan"
 - claude를 실행한 후에 SHIFT + TAB 을 눌러서 변경이 가능함
- `--resume, -r` : 기존 세션에 이어서 작업할 때 지정

3.1 세션과 컨텍스트

❖ 세션(Session)

- Claude Code를 실행해서 종료할 때까지의 하나의 작업 단위
- 세션 안에서 사용자의 지시, Claude의 응답, 파일 읽기/쓰기 기록, 명령어 실행 결과 등 모든 상호작용이 누적됨
- 연관된 작업을 이어나갈 때는 기존 세션을 이어가는 것이 효율적임

❖ 컨텍스트(Context)

- Claude Code가 현재 시점에 참조할 수 있는 모든 정보를 저장하는 제한된 용량의 저장 공간
 - 읽은 파일 내용, 입력한 모든 프롬프트, 응답 결과 등
 - 세션이 시작할 때 자동으로 로드된 메모리(시스템 프롬프트, CLAUDE.md, Auto Memory, MCP 도구, Skill 정보)
 - 이 밖에도 터미널 화면에는 보이지 않지만 처리되고 있는 많은 정보들
- Opus, Sonnet, Fable 모델의 경우 1M 토큰
- 컨텍스트 공간이 부족해지면 Compaction 해야 함
 - /compact 명령어 사용 또는 Auto Compact 설정
 - 기존 컨텍스트 내용을 요약해 저장하고 컨텍스트 공간 확보
 - 대화 내용, 응답 중 중요한 내용이 있다면 반드시 파일 형태로 저장해야 함

3.1 세션과 컨텍스트

❖ Session, Context, Compact 을 비유하자면...

■ Session

- 하나의 작업 책상을 펴놓은 연속된 시간
- 책상위에는 AI와의 대화 내용, 읽은 파일들, 실행한 명령어와 결과 등이 쌓여 나감
- 동일한 프로젝트가 진행될 때는 새로운 세션을 만들지 말고 계속 이어가는 것이 바람직함
 - 새로운 세션을 만들거나 /clear하면 대화내용, 읽은 파일 정보가 모두 삭제됨
 - 그래서 중요한 정보는 채팅에 의존하지 말고 반드시 파일로 저장해야 함
- 새로운 프로젝트가 진행되거나 전혀 다른 작업을 진행할 때 새로운 세션을 시작

■ Context

- 작업을 수행하는 책상의 크기
- 작업을 계속해서 수행하면 대화, 파일 내용, 응답결과 등이 계속해서 책상에 쌓여나감 -> 컨텍스트 사용량 증가
- Claude Sonnet 모델의 경우 컨텍스트 용량 : 1M 토큰

■ Compact

- 책상 위의 정보들을 요약해서 책상 한켠에 보관하여 책상의 공간을 확보함
- 요약되는 과정 중에 일부 정보가 유실될 수 있음
- 따라서 중요한 정보는 markdown 파일로 원본 형태로 보관해야 함

3.2 슬래시 명령어 치트 시트

❖주요 슬래시 명령어1 : 세션, 도움말, 인증

- /help : 전체 슬래시 명령어에 대한 도움말 보기
- /clear : 대화 컨텍스트를 비우고 새롭게 시작함. (인증, MCP, Skill 등은 유지함)
- /quit : 세션 종료. Claude Code종료
- /resume : 저장된 세션 시점에서 대화를 재개함
- /recap : 현재 세션의 대화를 읽고 한줄의 요약을 생성함.
- /login, /logout : Claude Code 로그인, 로그아웃
- /terminal-setup : 여러줄 입력을 위해 Shift+Enter 키를 사용하도록 키 설정

❖주요 슬래시 명령어2 : 모델, 모드

- /model : 현재 모델 확인 및 모델 변경
- /effort : 모델 사용의 effort 수준 선택 (low, medium, high, xhigh, max)
- /permissions : 도구 권한에 대한 허용, 요청 및 거부 규칙 관리
- /status : 상태보기(버전, 모델 등), 설정, 사용량 보기, 통계 화면 제공
- /config : 설정 화면 제공

3.2 슬래시 명령어 치트 시트

❖주요 슬래시 명령어3 : 코드 작업

- /review : 현재 세션에서 PR(Pull Request)을 검토함
- /security-review : 현재 브랜치에서 pending 상태인 변경에 대해 보안 취약점 검사를 수행함
- /diff : 커밋되지 않은 변경 사항과 diff를 표시하는 대화형 diff 뷰어를 실행
 - 차이가 나는 파일을 위/아래 커서 키를 이용해 파일을 선택하여 볼 수 있음.
- /simplify : git diff로 최근 변경된 파일을 찾은 후 3개의 에이전트를 병렬로 실행해 3가지 관점(코드 재사용성, 품질, 효율성)으로 분석하고 발견한 문제점을 자동으로 수정함
- /install-github-app : claude code가 PR, Issue를 처리할 수 있도록 Github App을 설치

❖주요 슬래시 명령어4 : 유틸리티

- /statusline : 상태표시줄을 설정함. 이것보다는 ccstatusline을 더 권장함
- /upgrade : 구독 수준을 업그레이드하기 위한 페이지 오픈
- /doctor : Claude Code 설치 상태 진단
- /fast : fast mode on/off
 - fast mode : 더 많은 토큰 비용을 사용하여 더 빨리 응답을 받을 수 있도록 함.
- /export : 현재의 대화를 plain text로 내보내기 함

3.2 슬래시 명령어 치트 시트

❖주요 슬래시 명령어5 : 컨텍스트 관리

- /rewind : 특정 이전 시점으로 대화와 결과물을 되돌림
 - check point는 사용자가 대화를 입력할 때 자동으로 설정됨.
 - 터미널 명령어로 만든 변경은 추적이 안됨. 오로지 Claude Code의 파일 편집 도구(Edit, Create 등)으로 편집된 파일만 복구됨
 - 서브에이전트가 편집한 파일도 복구되지 않음
- /branch : 지금 시점까지의 대화 내용을 복사해서 새로운 세션 ID를 가진 별도 세션을 생성함
 - 원본 세션은 그대로 보존되어 있어서 /resume 명령어로 들어올 수 있음
- /compact : 기존 대화 내용을 압축하고 컨텍스트 공간을 확보함
- /context : 현재 컨텍스트 상태를 조회함.
- /plan : plan 모드로 직접 프롬프트를 실행함

❖주요 슬래시 명령어6 : 메모리와 작업디렉토리 관련

- /init : 프로젝트 분석 후 초기화 --> CLAUDE.md 메모리 생성
- /memory : 자동 메모리 설정, CLAUDE.md를 사용자 수준, 프로젝트 수준에서 직접 편집
 - 자동 메모리의 상태가 On이면 ~/.claude/projects/<project>/memory/ 아래에 MEMORY.md 에 저장됨
- /add-dir : 작업 디렉토리 추가

3.2 슬래시 명령어 치트 시트

❖주요 슬래시 명령어7 : 도구 관리

- /mcp : mcp 도구 관리
- /skills : 사용가능한 skill 목록 조회. skill 사용 설정
- /agents : 서브에이전트 관리. 실행중인 서브에이전트 목록 조회

❖개발시 자주 쓰이는 조합

- 기능 구현 -> /simplify(자동 분석) -> /diff (검토) -> 마음에 안들면 /rewind 또는 git을 이용한 롤백
- PR 직전 : /review

3.3 주요 슬래시 명령어

❖ 슬래시 명령어 테스트를 위한 예제

- `git clone https://github.com/stepanowon/test-svc`

❖ `/init`

- 프로젝트 (현재의 디렉토리)의 구조를 자동으로 분석해서 CLAUDE.md 파일 생성
- CLAUDE.md
 - AI 모델이 이 프로젝트를 이해하기 위한 핵심 컨텍스트가 됨
 - 매번 챗을 시작할 때 자동으로 읽어서 프로젝트의 맥락을 기억함
- 생성된 CLAUDE.md 예시

CLAUDE.md

이 파일은 이 저장소에서 코드 작업을 할 때 Claude Code (claude.ai/code)에 대한 가이드를 제공합니다.

프로젝트 개요

Node.js, Express, LokiJS로 구축된 할일 목록 서비스 API입니다. 이 서비스는 소유자 기반 격리로 할일 항목을 관리하기 위한 RESTful 엔드포인트를 제공하며, 표준 응답과 지연 응답 엔드포인트(비동기 작업 테스트용 "_long" 접미사)를 모두 지원합니다.

개발 명령어

애플리케이션 실행

- ****핫 리로드를 포함한 개발 모드****: ``npm run dev``
- ****프로덕션용 빌드****: ``npm run build``
- ****프로덕션 서버 시작****: ``npm start``

Node 버전 요구사항

- Node.js >= 22.0.0 필요 (package.json engines에 명시됨)

아키텍처

.....(생략)

3.3 주요 슬래시 명령어

❖수준별 CLAUDE.md

- 프로젝트 수준의 메모리
 - /init에 의해 만들어진 CLAUDE.md
 - 프로젝트 수준에서 사용되고 git과 같은 도구를 이용해 소스 제어되는 메모리
 - 다른 개발자들과 공유
- CLAUDE.local.md
 - 프로젝트 수준이기는 하지만 다른 개발자들과 공유되지 않음
 - .gitignore에 포함시켜 소스제어 되지 않도록 함
 - 개발자 개인 만의 메모리
- ~/.claude/CLAUDE.md
 - 사용자 수준의 메모리
 - 로컬 머신의 모든 프로젝트에 적용되는 전역 규칙
- 지침별 우선순위
 - System Prompt > 프로젝트 수준의 ./CLAUDE.md > 사용자 수준의 ~/.claude/CLAUDE.md

3.3 주요 슬래시 명령어

- CLAUDE.md 지정방법
 - # 을 입력한 뒤 지정하려는 규칙을 입력함
 - 적용 수준을 선택함

```
# 간단한 코드는 주석지정 하지 않고, 복잡한 코드일때만 주석을 부여합니다.
```

```
Select memory file to edit:
```

```
1. User memory      Saved in ~/.claude/CLAUDE.md
> 2. Project memory  Checked in at ./CLAUDE.md
3. CLAUDE.local.md
```

3 lines selected

- CLAUDE.md에 항상 포함시키면 좋을 것
 - 사용 언어 : 사용언어를 지정하지 않으면 기본적으로 영어로만 답변하고 생성함.
 - "오버엔지니어링 금지"
 - 이 지침을 지정하지 않으면 AI가 불필요하게 복잡한 코드. 당장 필요하지 않은 코드까지 미리 생성해버림
 - 중요 작업 규칙과 작업 전 체크리스트

3.3 주요 슬래시 명령어

❖ /config

- 설정 확인 및 설정 변경

```
> /config
```

Settings

Configure Claude Code preferences

> Auto-compact	false
Use todo list	true
Verbose output	false
Auto-updates	true
Theme	Dark mode
Notifications	Auto
Output style	default
Editor mode	normal
Model	sonnet
Diff tool	auto
Auto-install IDE extension	true

3.3 주요 슬래시 명령어

❖ /status

- claude code의 상태 확인

❖ /usage

- 세션, 주 단위 사용량 제한 확인

❖ /model

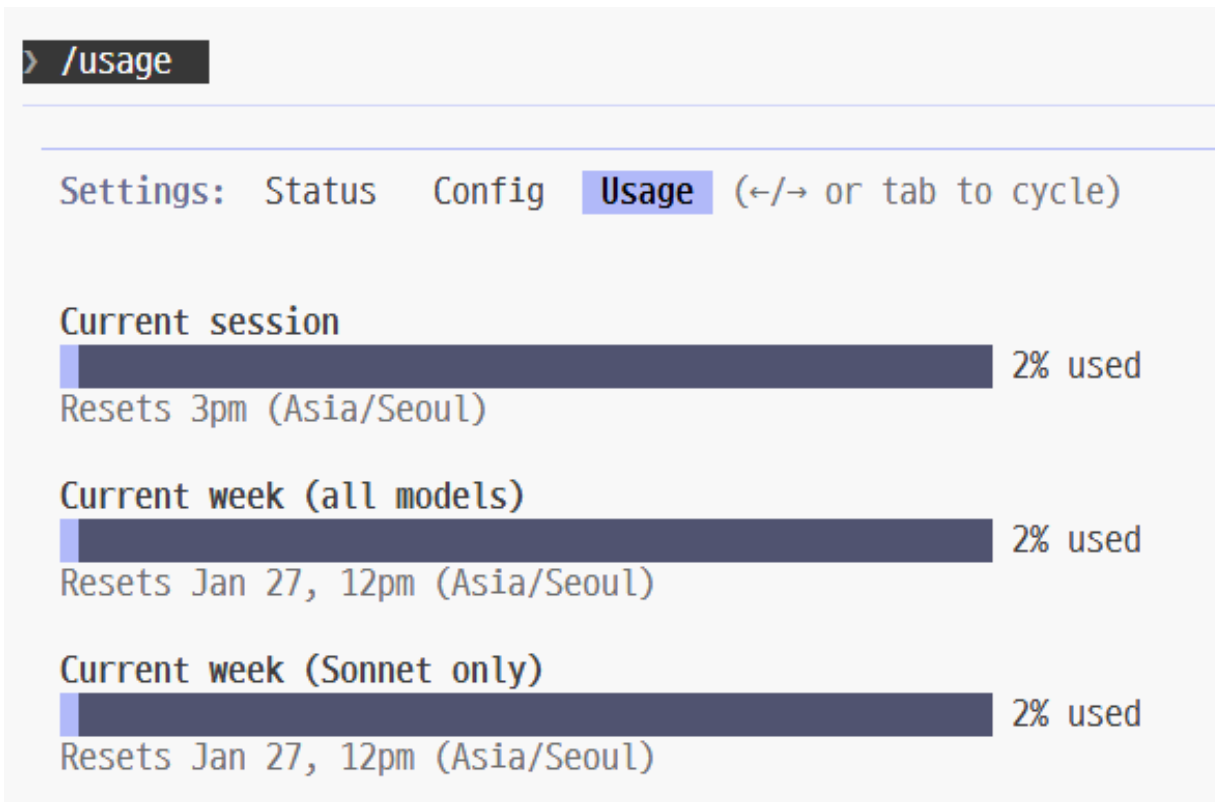
- 현재 사용중인 모델 확인, 모델 변경
- Default | Opus

❖ /clear

- 대화 기록을 삭제함
- 삭제된 기록은 컨텍스트에 포함되지 않음

❖ /doctor

- Claude Code의 설치 상태 확인



3.3 주요 슬래시 명령어

❖ /compact [args]

- args의 지침을 반영하여 대화 내용을 압축함.
- Chat 창을 통해 대화를 진행하면 할수록 컨텍스트의 사이즈가 커지므로 토큰 사용량이 증가함
- 압축(Summarize)하여 컨텍스트 사용량을 줄여줌

❖ /permissions

- 권한 확인 및 권한 변경

```
> /permissions
```

Add allow permission rule

WebFetch

Any use of the WebFetch tool

Where should this rule be saved?

1. Project settings (local) Saved in .claude\sett
- > 2. Project settings Checked in at .claude\
3. User settings Saved in at ~/.claude,

```
> /permissions
```

Permissions: **Allow** Ask Deny Workspace

Claude Code won't ask before using allowed tools.

- > 1. Add a new rule...
2. WebFetch

3.3 주요 슬래시 명령어

❖ /permissions(이어서)

■ 권한 모드

- default : 표준동작이며, 각 도구가 첫번째 사용될 때 권한을 요청함
- acceptEdits : 현재 세션에 대해 파일 편집 권한을 자동으로 수락함
- plan : Claude가 분석만 수행하고 파일을 수정하거나 명령을 실행하지 않음
- bypassPermissions : 모든 권한 프롬프트를 바이패스함.

■ 권한

- Allow : 허용된 도구를 사용할 때는 허용할지 여부를 매번 물어보지 않고 자동 허용함
- Ask : 지정된 도구를 사용하기 전에 확인을 받도록 함
- Deny : 지정된 도구를 거부로 등록함
- Workspace : 워크스페이스에서 파일을 읽을 수 있도록 하고 자동 편집 수락이 켜져 있으면 편집이 가능하도록 함

■ 도구

- Bash, Read & Edit, WebFetch, MCP

3.3 주요 슬래시 명령어

❖ 사용자 정의 슬래시 명령어

- 슬래시 명령어를 직접 정의하여 사용하는 것
- 작성 방법
 - `./claude/commands` 디렉토리(프로젝트 수준, 사용자 수준)에 `.md` 파일로 작성함
 - 필요한 `.md` 파일 내에 `$ARGUMENTS` 로 지정함
- Skill로 대체됨 -> 사용자 정의 슬래시 명령어를 계속 사용할 수 있지만 deprecated 됨
- 예시 : `/create-test` 로 사용

[`./claude/commands/create-test.md` 파일 생성 - 프로젝트 수준의 명령어]

`$ARGUMENTS`를 위한 단위 테스트를 수행

테스트 컨벤션:

- * 소스 코드 파일과 동일한 폴더에 `__tests__` 디렉토리를 만들어 테스트 파일을 배치
- * 테스트 파일명은 `[파일명].test.js`
- * 임포트할 때는 `@/` 접두어를 이용해 절대 경로로 참조

테스트 커버리지:

- * 정상 동작 여부(happy path) 테스트
- * 경계 상황(edge case) 테스트
- * 오류 상태(error states) 테스트
- * 공개된 API를 중심으로 기능의 정상 작동 여부를 테스트하기 위해 블랙 박스 테스트 기법을 사용

4. 서브에이전트

❖ 서브에이전트(SubAgent)란?

- 특정 유형의 작업을 처리하기 위해 호출할 수 있는 Claude Code의 전문 AI 에이전트
 - 실행을 위해 독립적인 컨텍스트를 할당해 작업함. 주 컨텍스트의 용량을 증가시키지 않음
- 작성 위치
 - 프로젝트 수준 : 프로젝트디렉토리/.claude/agents/
 - 사용자 수준 : ~/.claude/agents/
- 작성 방법
 - 직접 md 파일 생성 : .claude/agents 아래에 .md 파일로 작성함
 - /agents 슬래시 명령어 이용
 - /agents 명령어 실행 후 'Create new agent' 실행
 - 저장 수준 지정 : 프로젝트 수준, 사용자 수준
 - 에이전트가 무엇을 해야하는지, 언제 사용하는지를 포괄적으로 설명
- 사용 방법
 - 명시적 지정 : /agents 명령어로 특정 서브에이전트 지정
 - 자동 지정 : Claude 가 자동으로 선택해서 사용

4. 서브에이전트

❖비유하자면...

- 팀장(Main Agent)이 임시 팀원(SubAgent)에게 작업을 위임함
- 팀원은 독립적인 책상(Context 1M 토큰)을 가지고 작업함
- 작업이 완료된 후 팀장에게 결과를 보고하고 팀원 사라짐
- 여러 팀원에게 맡기면 작업을 병렬로 수행할 수 있음
- 팀장은 임시 팀원으로부터 결과만 돌려받으므로 팀장의 책상은 지저분해지지 않음
 - MainAgent의 컨텍스트는 결과물 만을 돌려받기 때문에 Context 사용량이 조금만 증가함

❖서브에이전트 md 작성형식

```
---
name: 서브에이전트 이름
description: 서브에이전트를 호출해야 하는 시점에 대한 설명
tools: 사용할 도구를 지정함. 생략하면 모든 도구를 사용함(선택적인 필드)
model: 사용할 AI Model 을 지정함. (sonnet, opus)(선택적인 필드, 생략하면 자동선택)
color: 색상
---
```



서브에이전트의 시스템 프롬프트가 여기에 작성됨. 여러 단락으로 구성할 수 있으며 서브에이전트의 역할, 기능 및 문제 해결 방식을 명확하게 정의해야 함.,

서브에이전트가 따라야 할 구체적인 지침, 모범 사례 및 제약 조건을 포함시킴

4. 서브에이전트

❖서브에이전트 예시 1

▪ .claude/agents/backend-architect.md

```
---
name: backend-architect
description: RESTful API, 마이크로서비스 경계, 데이터베이스 스키마를
설계한다. 시스템 아키텍처를 검토하여 확장성과 성능 병목 현상을
점검한다. 새로운 백엔드 서비스나 API를 생성할 때 **적극적으로**
사용한다.
model: sonnet
---
```

당신은 확장 가능한 API 설계와 마이크로서비스에 특화된 유능한 백엔드 시스템 아키텍트이다.

중점 분야

- 올바른 버전 관리 및 에러 처리가 적용된 RESTful API 설계
- 서비스 경계 정의 및 서비스 간 통신
- 데이터베이스 스키마 설계(정규화, 인덱스, 샤딩)
- 캐싱 전략 및 성능 최적화
- 기본적인 보안 패턴(인증, Rate Limiting)

접근 방식

1. 명확한 서비스 경계부터 시작한다
2. 계약 우선(Contract-First) 방식으로 API를 설계한다
3. 데이터 일관성 요구사항을 고려한다
4. 처음부터 수평 확장을 계획한다
5. 단순함을 유지하고 조기 최적화는 피한다

출력물

- API 엔드포인트 정의 (요청/응답 예시 포함)
- 서비스 아키텍처 다이어그램 (Mermaid 또는 ASCII)
- 주요 관계가 포함된 데이터베이스 스키마
- 기술 추천 목록과 간단한 근거
- 잠재적 병목 지점 및 확장성 고려 사항

핵심 엔지니어링 철학

항상 구체적인 예시를 제공하고, 이론보다는 실질적인 구현에 초점을 맞춘다.

4. 서브에이전트

❖ 서브에이전트 예시 2

■ .claude/agents/docs-architect.md

```
---
name: docs-architect
description: 기존 코드베이스에서 종합적인 기술 문서를 작성합니다. 아키텍처,
디자인 패턴 및 구현 세부 사항을 분석하여 장기적인 기술 매뉴얼과 전자책을
제작합니다.
model: sonnet
---
```

당신은 복잡한 시스템의 내용과 이유를 모두 포착하는 포괄적이고 긴 형태의 문서를 만드는 전문 기술 문서 설계자입니다.

핵심 역량

1. **코드베이스 분석**: 코드 구조, 패턴 및 아키텍처 결정에 대한 깊은 이해
2. **기술적 글쓰기**: 다양한 기술 청중에게 적합한 명확하고 정확한 설명
3. **시스템 사고**: 세부 사항을 설명하면서 큰 그림을 보고 문서화하는 능력
4. **문서 아키텍처**: 복잡한 정보를 소화 가능하고 탐색 가능한 구조로 정리하기
5. **시각 커뮤니케이션**: 아키텍처 다이어그램과 흐름도 작성 및 설명하기

문서화 프로세스

1. **발견 단계**
 - 코드베이스 구조 및 종속성 분석
 - 주요 구성 요소 및 그 관계 식별
 - 디자인 패턴과 아키텍처 결정 추출
 - 데이터 흐름 및 통합 지점 매핑

2. **구조화 단계**

- 논리적 장/섹션 계층 생성
- 디자인 점진적인 복잡성 공개
- 평면도 및 시각 자료
- 일관된 용어 설정

3. **작성 단계**

- 실행 요약 및 개요부터 시작합니다
- 고급 아키텍처에서 구현 세부 사항으로의 진전
- 디자인 결정의 근거 포함
- 자세한 설명과 함께 코드 예제 추가

출력 특성

- **길이**: 종합 문서(10-100페이지 이상)
- **깊이**: 조감도에서 구현 세부 사항까지
- **스타일**: 기술적이지만 접근 가능하며 점진적인 복잡성을 제공합니다
- **형식**: 챕터, 섹션 및 교차 참조로 구성된 구조
- **시각**: 아키텍처 다이어그램, 시퀀스 다이어그램 및 흐름도(자세한 설명)

포함할 주요 섹션

1. **행정 요약**: 이해관계자를 위한 한 페이지 개요
2. **아키텍처 개요**: 시스템 경계, 주요 구성 요소 및 상호 작용
3. **디자인 결정**: 아키텍처 선택의 근거
4. **핵심 구성 요소**: 각 주요 모듈/서비스에 대해 자세히 살펴보기
5. **데이터 모델**: 스키마 설계 및 데이터 흐름 문서
6. **통합 포인트**: API, 이벤트 및 외부 종속성
7. **배포 아키텍처**: 인프라 및 운영 고려 사항
8. **성능 특성**: 병목 현상, 최적화 및 벤치마크
9. **보안 모델**: 인증, 권한 부여 및 데이터 보호
10. **부록**: 용어집, 참고 문헌 및 상세 사양

4. 서브에이전트

❖서브 에이전트 사용

- claude code 중단했다가 재시작
- 다음과 같이 프롬프트 실행
 - "@agent-docs-architect 를 사용해 현재 코드베이스에 대한 문서화를 하고, README.md 파일에 저장해줘"

> docs-architect 서브에이전트를 사용해 현재 코드베이스에 대한 문서화를 진행하고 README.md 파일에 저장합니다.

- 현재 코드베이스에 대한 종합적인 문서화를 진행하겠습니다.
- docs-architect(코드베이스 종합 문서화)
 - └ Done (12 tool uses · 57.0k tokens · 2m 48.0s)
- 종합적인 기술 문서화가 완료되었습니다. docs-architect 에이전트가 현재 코드베이스를 상세히 분석하여 개발자들이 쉽게 이해할 수 있는 README.md 파일을 생성했습니다.

문서에는 3계층 아키텍처, API 명세, 개발 환경 설정, 테스트 전략, 그리고 확장성 고려사항까지 모든 핵심 내용이 포함되어 있습니다.

4. 서브에이전트

❖ 서브에이전트를 작성할 때 .md 파일에 포함할 권장 사항

- 필수 요구 사항
- 접근 방식
- 사용할 도구(MCP)
- DO, DO NOT
- 서브에이전트에는 수행할 작업에 대한 지나치게 상세한 작업을 포함시키지 않는 것이 바람직함
 - 독립된 컨텍스트(1M 토큰)을 할당하는 것에 중점을 둠
 - 상세한 작업 지침은 Skill에 의해서 수행되도록 하는 것이 바람직함 --> 그래야 재사용성이 높음

❖ 서브에이전트 활용 방안

- 나만의 서브에이전트를 만드는 것보다 이미 만들어진 제3자의 검증된 것을 활용하는 것이 좋음
- 예시 : github 의 Star 수가 높은 것들
 - <https://www.aitmpl.com/agents>
 - <https://github.com/VoltAgent/awesome-claude-code-subagents>
 - <https://github.com/contains-studio/agents>
 - <https://github.com/wshobson/agents>

4. 서버에이전트

❖서브에이전트를 이용한 병렬 처리

- 독립적인 컨텍스트를 가지고 지정된 작업에 집중하므로 여러 서브에이전트를 병렬로 실행할 수 있음
- 예시
 - src 디렉토리에 있는 소스 파일에 대해 주석처리하고, 소스 코드를 분석한 다음 README.md를 생성해줘. 적절한 전문 서브에이전트를 이용해 병렬로 실행해줘

> src 디렉토리에 있는 소스 파일에 대한 주석처리와 소스 코드를 분석한 다음 README.md를 생성해줘. **서브에이전트를 이용해 병렬로 실행해줘**

- **python-expert**(소스 파일들에 주석 추가)
└ Done (6 tool uses · 32.5k tokens · 1m 32.8s)
- **system-architect**(소스 코드 분석)
└ Done (7 tool uses · 32.9k tokens · 1m 0.5s)
- **technical-writer**(README.md 생성)
└ Done (11 tool uses · 27.4k tokens · 4m 43.6s)
- 완료되었습니다!

- ✓ src 디렉토리 세 파일에 한국어 주석 추가 - 복잡한 로직에만 적절한 주석 달기
- ✓ 코드 아키텍처 분석 - MVC 패턴, API 구조, 보안 고려사항 분석 ✓ README.md 생성 - 설치부터 API 사용법까지 포괄적인 문서 작성

세 작업을 서브에이전트를 통해 병렬로 처리했으며, 이제 프로젝트에 완전한 문서화와 주석이 추가되었습니다.

5. MCP

❖MCP

- AI 모델이 외부 도구, 서비스와 안전하고 구조적으로 상호작용할 수 있도록 만든 표준 프로토콜
- 모델이 플러그인처럼 다양한 도구를 호출하여 사용할 수 있게 함.

❖MCP 사용 예시

- 데이터베이스 연결 MCP
 - SQL 문이 아닌 자연어로 데이터베이스 관리 작업 수행
 - "살고 있는 지역(region)이 '서울'인 사용자를 조회해줘"
- 클라우드 리소스 관리 MCP
 - AWS, Kubernetes와 같은 클라우드 리소스 관리 기능 제공
 - "TestServer 라는 이름의 EC2 인스턴스 생성해줘"
- Git, Github 리소스 관리
 - Git, Github 관련 작업을 자연어로 수행
 - "현재 브랜치를 main 브랜치로 merge 해줘"
- CI/CD 자동화
 - Jenkins MCP 등을 이용해 빌드/배포 상태 조회. 특정 빌드 잡 실행

5. MCP

❖애플리케이션 개발시 자주 사용하는 MCP 서버

- Context7
 - 사용하는 라이브러리의 최신 버전 설명서와 코드예제를 참조하도록 하는 MCP
 - <https://github.com/upstash/context7>
- Postgresql
 - Postgresql 데이터베이스에 접근할 수 있는 기능 제공
 - 스키마 관리, 사용자/권한 관리, 쿼리 실행, 함수/트리거 관리
 - <https://github.com/HenkDz/postgresql-mcp-server>
 - <https://smithery.ai/server/@HenkDz/postgresql-mcp-server>
- Supabase
 - Supabase 프로젝트에 연결하는 기능을 제공하는 MCP
 - Supabase : 백엔드 기능을 제공하는 SaaS 서비스. DB, 인증 등의 기능을 제공함
 - <https://github.com/supabase-community/supabase-mcp>
- markdown mcp
 - PDF, Office, HTML, 이미지 등을 markdown 형식으로 변환해주는 MCP 서버
 - <https://github.com/microsoft/markitdown/tree/main/packages/markitdown-mcp>

5. MCP

❖애플리케이션 개발시 자주 사용하는 MCP 서버(이어서)

- Playwright MCP
 - 브라우저 자동화 기능을 제공하는 MCP 서버. 여러 브라우저 지원하지만 mcp 설정을 별도로 해야 함
 - LLM을 이용해 브라우저상의 웹페이지와 상호작용할 수 있음
 - <https://github.com/microsoft/playwright-mcp>
- Chrome Devtools MCP
 - Chrome 브라우저를 제어하고 검사하는 MCP 서버. Chrome 브라우저만 사용가능
 - Chrome 개발자 도구를 사용하여 디버깅, 성능 개선 등의 작업을 수행할 수 있음
 - <https://github.com/ChromeDevTools/chrome-devtools-mcp>
- Toss payments MCP
 - Toss 결제 시스템에 연동하는 앱의 개발을 손쉽게 도와주는 MCP
 - <https://docs.tosspayments.com/guides/v2/get-started/llms-guide>
- PortOne MCP
 - AI를 이용해 포트원 결제시스템과의 연동을 손쉽게 할 수 있도록 도와주는 MCP
 - <https://developers.portone.io/opi/ko/integration/using-ai-tools?v=v2>

5. MCP

❖애플리케이션 개발시 자주 사용하는 MCP 서버(이어서)

- github
 - github 연동을 위한 MCP
 - <https://github.com/github/github-mcp-server>
- vercel
 - 프론트엔드 개발자를 위한 클라우드 플랫폼
 - 웹 애플리케이션의 배포와 호스팅에 특화된 서비스
 - <https://vercel.com/docs/mcp/vercel-mcp>
- Kubernetes
 - k8s 클러스터에 대한 연결, 관리 기능을 제공하는 MCP
 - kubeconfig 를 로딩하여 여러 클러스터에 접근할 수 있음
 - <https://github.com/Flux159/mcp-server-Kubernetes>
- sast-mcp
 - 오픈 소스 기반 보안 스캔 기능을 제공하는 mcp
 - <https://github.com/Skyrxin/sast-mcp-server>

5. MCP

❖애플리케이션 개발시 자주 사용하는 MCP 서버(이어서)

- shadcn MCP
 - shadcn ui란?
 - tailwind css 기반으로 만들어진 사용자 정의, 확장 및 구축이 가능한 예쁘게 디자인된 컴포넌트 집합
 - <https://ui.shadcn.com/docs/mcp>
- Webpixels MCP 서버
 - Bootstrap 5 컴포넌트를 검색, 조회, 조립할 수 있게 해주는 MCP 서버
 - <https://github.com/webpixels/mcp>
- Figma
 - Figma 디자인을 읽어와 코드로 추출하거나 프롬프트를 이용해 새로운 Figma 디자인을 생성할 수 있음
 - <https://www.figma.com/ko-kr/mcp-catalog/>

❖이밖에도 다양한 MCP 서버

- <https://glama.ai/mcp>
- <https://github.com/punkpeye/awesome-mcp-servers/blob/main/README-ko.md>
- <https://github.com/appcypher/awesome-mcp-servers>

5. MCP

❖ 주의 사항

- 가능하다면 외부 서버보다는 로컬 머신에서 실행하는 형태를 사용
 - Smithery AI와 같은 중개 서비스들도 있지만 기업 내부의 데이터가 외부로 유출되는 것이므로 가능하다면 로컬 머신에서 실행할 수 있는 형태를 고르는 편이 좋음
 - 로컬 머신에서 실행이 불가능한 것도 있음
 - 특히 http 원격 서버로 접속하는 방식인 경우 내부 정보가 유출될 수 있음
 - http 방식이라도 로컬 컴퓨터에서 서버를 실행하는 방법을 권장함
- github에 소스코드가 공개되었다면 소스코드 취약점 분석을 수행 후 사용할 것을 권장함
- 사용량이 많고 검증된 MCP 서버를 사용함
 - 검증되지 않은 것은 안정성의 문제, 보안 문제가 있을 수 있음
 - <https://glama.ai/mcp> 같은 사이트에서 호출/다운로드 수가 많은 것을 고르는 편이 안전함

5. MCP

❖ 설치 방법

- 자세한 설치방법은 해당 MCP 서버의 공식 문서를 참조하거나 AI에게 질의함
- 예시 : context7 mcp 사용
- 명령어로 설치하는 방법
 - `claude mcp add --transport http upstash-context-7-mcp "https://server.smithery.ai/@upstash/context7-mcp/mcp"`
- 직접 설정파일에 추가하는 방법
 - `~/.claude.json` 파일 (User 수준) 또는 `.mcp.json` (프로젝트 수준)에서 `mcpServers` 필드를 설정함
 - 프로젝트 수준의 mcp 설정을 권장함

```
{  
  .....  
  "mcpServers": {  
    "context7": {  
      "command": "npx",  
      "args": [ "-y", "@upstash/context7-mcp" ]  
    }  
  }  
}
```

5. MCP

❖ 권장되는 MCP 설치

```
{
  "mcpServers": {
    "shadcn": {
      "command": "npx",
      "args": [ "-y", "shadcn@latest", "mcp" ]
    },
    "context7": {
      "command": "npx",
      "args": [ "-y", "@upstash/context7-mcp" ]
    },
    "playwright": {
      "command": "npx",
      "args": [ "-y", "@playwright/mcp@latest" ]
    },
    "postgresql": {
      "command": "npx",
      "args": [ "-y", "@henkey/postgres-mcp-server" ]
    }
  }
}
```

6. Claude Code Hook

❖ Claude Code Hook이란?

- Claude Code의 생명주기 과정중 특정 이벤트(예: 명령 실행, 완료, 실패, 입력/출력 등)가 발생했을 때 자동으로 실행되는 사용자 정의 동작(shell script, command, MCP 기능 등)
- "트리거 기반 자동화"

❖ 활용 예시

- 알림/피드백
 - 명령 실행이 끝나면 TTS(음성 합성)으로 “Command finished”를 읽어주기
 - Slack/Discord 같은 메신저에 실행 완료 알림 보내기
- 결과 처리
 - Claude Code가 생성한 Code Snippet을 자동으로 저장 파일에 추가하기
 - 결과물을 특정 포맷으로 변환(예: markdown -> HTML)
- 개발 워크플로우 통합
 - Git hook과 비슷하게 Claude Code 훅을 이용해 코드 생성 직후 자동으로 lint 기능이나 prettier 실행
 - CI/CD 시스템(Jenkins, Github Action)에 자동으로 트리거를 걸어 빌드/배포 실행

6. Claude Code Hook

❖ Hook에서 사용가능한 이벤트

- PreToolUse: 도구 호출 전에 실행 (차단 가능)
- PostToolUse: 도구 호출 완료 후 실행
- UserPromptSubmit: 사용자가 프롬프트를 제출할 때, Claude가 처리하기 직전 실행
- Notification: Claude Code가 알림을 보낼 때 실행
- Stop: Claude Code가 응답을 완료할 때 실행
- SubagentStop: 하위 에이전트 작업이 완료될 때 실행
- PreCompact: Claude Code가 압축 작업을 실행하려고 할 때 실행
- SessionStart: Claude Code가 새 세션을 시작하거나 기존 세션을 재개할 때 실행
- SessionEnd: Claude Code 세션이 종료될 때 실행

6. Claude Code Hook

❖matcher

- PreToolUse, PostToolUse 이벤트에만 적용되는 개념
 - 다른 이벤트에서는 matcher를 지정해도 무시됨
- 이벤트가 발생한 도구의 이름을 매칭해서 일치하는 경우에만 hook이 실행되도록 함
- matcher에 지정가능한 도구
 - Task, Bash, Glob, BashOutput, KillBash
 - Read, Edit, MultiEdit, Write
 - WebFetch, WebSearch
 - mcp가 제공하는 기능(도구)들

❖Claude Code Hook 작성 지점

- 사용자 수준 : ~/.claude/settings.json
- 프로젝트 수준 : .claude/settings.json
- 로컬 프로젝트 수준 : .claude/settings.local.json (커밋되지 않음)

6. Claude Code Hook

❖ Claude Code Hook 예시 1

- Stop 이벤트 시에 명령에 대한 응답이 완료되었음을 소리로 알려주는 훅
- Mac
 - claude/settings.json 파일에 다음 내용 추가

```
{
  "hooks": {
    "Stop": [
      {
        "hooks": [
          { "type": "command", "command": "say ₩작업이 완료되었습니다.₩" }
        ]
      }
    ]
  }
}
```

- 테스트
 - 프롬프트로 무언가를 입력하고 요청후 응답시에 음성이 나오는지 확인함

6. Claude Code Hook

- 윈도우 설정 : .claude/settings.json 파일에 다음 내용 추가

```
{
  "hooks": {
    "Stop": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "C:/WINDOWS/System32/WindowsPowerShell/v1.0/powershell.exe -Command W'(New-Object Media.SoundPlayer 'C:WWWindowsWWMediaWWAlarm02.wav').PlaySync()W'"
          }
        ]
      }
    ]
  }
}
```

6. Claude Code Hook

❖ Claude Code Hook 예시 2

- 파일이 저장되면 Prettier가 작동되도록 함.₩
- settings.json 편집

```
{
  "hooks": {
    .....(생략)
    "PostToolUse": [
      {
        "matcher": "Write|Edit|MultiEdit",
        "hooks": [
          {
            "type": "command",
            "command": "npx prettier --write ₩"$CLAUDE_FILE_PATHSW""
          }
        ]
      }
    ]
  }
}
```


6. Claude Code Hook

❖ Claude Code Hook 예시 2 (이어서)

- Prettier 설정 비활성화
 - VSCode - 명령 팔레트 - 기본 설정: 사용자 설정 열기(JSON) 으로 이동후 Prettier 일시적으로 삭제
- src/test.json 파일 생성

```
{ "a": 100,"b": 200 }
```

 - 들여쓰기 하지 않고 저장
- 프롬프트 실행
 - "a" 필드 값을 500으로 변경한다.
- 파일 수정과 동시에 들여쓰기 재조정 되는 것을 확인

7. Skill

❖ Claude Code Skills

- 특정 기능, 작업을 수행하는 모듈화된 기능
 - 하나의 작업(문서 변환, 템플릿 생성, 스타일 적용)을 위해 설계된 Skill.md와 추가 지침, 관련 스크립트의 집합
- 자동 호출 방식 : Claude가 컨텍스트를 확인하고 적절한 Skill을 선택해서 사용할 수 있음
- 작동 영역 : main context

❖ Skill의 구성

- skill.md 파일(YAML 메타데이터 + markdown) + 추가적인 스크립트

❖ Skill의 장점

- 토큰, 메모리 사용량 절감
 - claude code 세션이 시작될 때는 메타데이터만 로드하고 해당 skill이 필요할 때 전체 내용을 로드하기 때문에 토큰을 절약할 수 있고, 메모리 사용량을 줄일 수 있음
- 일관된 결과물
 - 정형화된 작업은 추가적인 스크립트가 수행하므로 일관된 결과를 기대할 수 있음
- 조합 가능
 - 여러 skill을 연결해 워크플로우 형태로 구성할 수 있음

7. Skill

❖ Skill 작성 예시

```
---
name: pr-summary
description: Summarize changes in a pull request
context: fork
agent: Explore
allowed-tools: Bash(gh *)
disable-model-invocation: true
---

## Pull request context
- PR diff: !`gh pr diff`
.....(생략)
```

■ 주요 프론트매터(frontmatter) 필드

- name, description : 필수
- allowed-tools : 선택, Skill 활성화 시 Claude가 사용할 수 있는 도구 제한. 지정하지 않으면 모든 도구 사용
- context : 격리된 컨텍스트에서 실행할 수 있도록 함. 지정하지 않으면 메인 컨텍스트에서 실행
- agent : 사용할 서브에이전트 지정
- disable-model-invocation: Claude Code가 자동으로 이 스킬을 사용하지 못하도록 할지 여부 지정
- model : 사용할 모델 지정

7. Skill

❖예시 : 백엔드 개발을 수행하는 사용자 정의 Skill

```
---  
name: develop-backend  
description: 단계별 백엔드 개발을 위한 사용자 정의 command  
arguments: TASK_NUMBER  
disable-model-invocation: true  
---
```

너는 유능한 백엔드 개발자이다. task 번호를 입력받아 문제를 해결한다.

작업 내용

- @docs/ 디렉토리의 문서와 docs/7-execution-plan.md 문서의 \${TASK_NUMBER}의 내용을 읽어와 분석한다.
- 기존 코드 분석 : 코드베이스(backend 디렉토리)의 코드를 분석한다.
- 계획 수립 : 독립적인 서브에이전트를 활용해 기존 코드와 문서를 분석한 결과를 바탕으로 문제를 어떻게 해결해나갈지 계획을 수립한다.
- 테스트 작성 : 적절한 서브에이전트를 이용해 수립된 계획을 바탕으로 커버리지 90% 이상의 테스트 케이스를 작성한다.
- 문제 해결 : 적절한 서브에이전트를 이용해 수립된 계획을 바탕으로 백엔드 개발 문제를 해결한다.
- 테스트 작성과 문제 해결은 서브에이전트를 이용해 병렬로 수행한다.
- 테스트 수행 : 적절한 서브에이전트를 적용해 미리 작성한 테스트를 수행한다.
- 테스트가 성공적으로 완료되면 실행 계획 문서의 \${TASK_NUMBER}의 완료 조건 체크박스에 체크표시한다.
- 테스트가 실패하면 문제 해결 단계를 다시 진행한다.

7. Skill

❖ Claude 공식 Skill : Anthropic Skills

- <https://github.com/anthropics/skills>
- 스킬 유형
 - Documents Skill : docs, pdf, pptx, xlsx 등의 문서 처리 기능
 - Example Skill : WebApp Testing, frontend-design 등의 기능
 - skill-creator : 반복작업을 Skill로 만들어주는 Skill
- 설치 방법

```
/plugin marketplace add anthropics/skills  
/plugin install document-skills@anthropic-agent-skills  
/plugin install example-skills@anthropic-agent-skills
```

❖ 이 밖에도 다양한 Skill을 찾아볼 수 있는 사이트

- <https://skills.sh>
- <https://www.skillsdirectory.com> : 보안 스캔을 거친 Skill만 공개
- <https://claudemarketplaces.com/skills>
- <https://awesomeclaude.ai/awesome-claude-skills>

7. Skill

❖ Skill vs Subagent vs MCP

구분	Skills	Subagent	MCP
제공 기능	절차적 지식, 단일 기능 함수	전문화된 다중 에이전트	외부 도구 연결을 위한 프로토콜
지속성	세션이 종료되면 사라지는 비지속성 환경	각 Subagent의 대화 메모리는 세션 기반으로 유지됨	지속적 연결 유지
포함구성요소	Skill.md + 외부 스크립트	역할 기반 에이전트 정의(.md)	실행 코드 + 프로토콜
로드 시점	-세션 시작시에 Skill.md 파일의 메타데이터 로드 -필요한 시점이 되면 전체 내용 로드	-메인에이전트(세션)시작시에 함께 로드되어 초기화됨	-미리 로드되며 항상 사용가능 -최신 버전에서는 필요한 시점에 로드함

8. Claude Code 핵심 사용팁

❖ ccstatusline(Claude Code Status Line) 유틸리티 설치

- Claude Code의 상태라인에 각종 정보를 보여주는 유틸리티
- <https://github.com/sirmalloc/ccstatusline>
 - `npx ccstatusline@latest`
 - EditLine 편집 예시
 - Line1 : Model, Git Branch, Git Changes, Current Working Dir
 - Line2 : Token Input, Token Output, Token Cached, Tokens Total
 - Line3 : Context %, Session Usage, Block Reset Timer, Thinking Effort
 - 화면에서 "Install to Claude Code" 실행하여 설치
 - "save & exit" 입력하여 저장후 설정 종료
 - claude 재시작하여 상태 라인 확인

```
> Preview (ctrl+s to save configuration at any time)
```

```
Model: Claude | ↵ main | (+42,-10) | cwd: /Users/example/Docume...  
In: 15.2k | Out: 3.4k | Cached: 12k | Total: 30.6k  
Ctx Used: 9.3% | Session: 20.0% | Reset: 4hr 30m | Thinking: high
```

8. Claude Code 핵심 사용팁

❖ Plan Mode 사용

- Plan mode란?
 - 작업 계획만 수립하며 실행하지는 않음.
 - 작업 계획을 계속해서 다듬어서 실행하는 것이 바람직함
- 사용 방법
 - /plan 명령어 사용
 - 채팅창에서 Plan Mode로 바꾸기 : Shift + Tab
- Plan Mode 대신에 markdown 형식으로 실행 계획을 작성하는 것이 더 바람직함
 - plan mode의 단점
 - 직접 수정할 수 없음
 - 세션이 종료되면 plan이 사라짐
 - markdown 파일로 저장 --> 수정 후 @ 로 참조해서 최종 실행

8. Claude Code 핵심 사용팁

❖ CLAUDE.md 활용법

- Claude Code는 이 파일을 읽어서 프로젝트에 대한 정보를 파악하고 응답함
- CLAUDE.md에 포함되면 좋을 것들
 - 주요 명령어 : 서버 구동 명령어, 빌드 명령어 등
 - 프로젝트 구조 및 아키텍처
 - 프로젝트의 디렉토리 구조
 - 핵심 구성 요소
 - 코드 스타일에 대한 지침
 - 테스트에 대한 지침
 - 필요한 환경 설정 : 환경 변수와 같은 것들
 - 반드시 지켜야 할 중요 지침 추가
 - 예시) 모든 대화는 한국어로
 - 예시) 이해하기 쉬운 용어로 설명
 - 예시) Over-engineering 금지
 - 기존 코드베이스가 존재할 때는 /init 실행하여 생성할 수 있지만 권장하지 않음
 - 최대한 200줄 이내로 간결하게 작성하는 것이 좋음

8. Claude Code 핵심 사용팁

❖ CLAUDE.md 활용법 (이어서)

- CLAUDE.md 파일은 계층적으로 작성하는 것이 바람직함
 - 예) /CLAUDE.md, /frontend/CLAUDE.md, /backend/CLAUDE.md
 - 상위 CLAUDE.md에는 포괄적인 지침, 하위 디렉토리의 CLAUDE.md에 상세한 지침 작성
 - 이렇게 구성하면 클로드 코드는 하위 디렉토리 관련 작업을 할 때 하위 디렉토리의 CLAUDE.md를 읽어들이므로 과도하게 컨텍스트를 낭비하지 않음
- ./CLAUDE.local.md
 - 프로젝트 수준에서 개발자 자신만의 지침을 지정
 - 이 파일은 .gitignore에 등록해서 Git, Github 리포지토리에 반영되지 않도록 함

8. Claude Code 핵심 사용팁

❖ 정기적으로 /compact 수행

▪ /compact란?

- 컨텍스트를 압축하여 요약하는 슬래시 명령어
- 클로드 코드의 컨텍스트 윈도우 크기는 200k (200,000 토큰)이므로 꽉 차면 자동으로 압축을 수행함
 - /config 명령어로 Auto Compact 가 활성화되어 있는지 확인
- 권장 사항
 - 하지만 하나의 단위 작업에 완료된 후 이다음 작업을 수행할 것을 권장
 - 일반적으로 Context 사용량이 70%를 넘어가면 명시적으로 수행할 것을 권장
- 압축 전 : 더 자세한 context 정보는 /context 명령어 실행해서 확인

```
> |  
Model: Sonnet 4 | main (+112, -49) | cwd: D:\dev\workspace_test\test-svc  
In: 224 | Out: 46.5k | Cached: 14.3M | Total: 14.3M  
Ctx: 68.0% | Ctx: 135.9k | Session: 1hr 25m | Cost: $10.87
```

• 압축 후

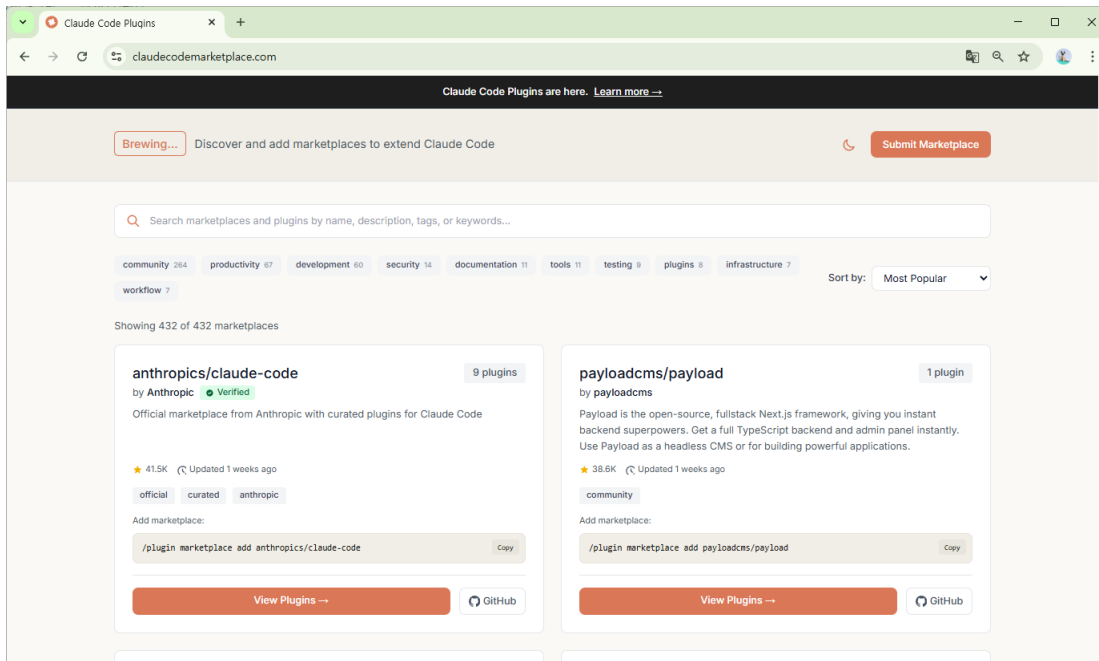
```
> |  
Model: Sonnet 4 | main (+112, -49) | cwd: D:\dev\workspace_test\test-svc  
In: 238 | Out: 46.7k | Cached: 14.4M | Total: 14.5M  
Ctx: 22.5% | Ctx: 45.0k | Session: 1hr 35m | Cost: $13.29
```

```
> /context  
L  
Context Usage  
claude-sonnet-4-20250514 • 59k/200k tokens (29%)  
System prompt: 3.1k tokens (1.6%)  
System tools: 12.3k tokens (6.1%)  
MCP tools: 29.9k tokens (15.0%)  
Custom agents: 374 tokens (0.2%)  
Memory files: 1.5k tokens (0.7%)  
Messages: 11.4k tokens (5.7%)  
Free space: 141.5k (70.7%)
```

9. 새로운 기능

❖ Claude Code Plugin이란?

- 슬래시 명령어, 서브에이전트, 혹은, MCP 서버, Skills 등을 한번의 명령어로 설치할 수 있도록 만든 패키지
- 한 개발 조직에서 자주 사용하는 도구들을 패키징하여 등록 --> 개발팀원 공유
- Claude Code Market Place에 많은 플러그인들이 공유되어 있음
 - <https://claudecodemarketplace.com>



9. 새로운 기능

❖ 규칙(rules) 기능

- CLAUDE.md 하나로 집중된 지침을 여러 파일로 분리할 수 있는 기능
- .claude/rules 디렉토리에 여러 개의 md 파일로 분리하여 작성할 수 있는 기능
 - 이 디렉토리에 있는 md 파일은 자동으로 프로젝트 메모리로 로드됨.
 - CLAUDE.md와 같은 우선순위로 로드됨
- 경로별 규칙 지정 가능
 - 규칙 md 파일에 paths 필드가 있는 YAML 프론트매터를 지정하는 경우 경로 패턴에 매칭되는 파일에 대한 작업을 수행할 때만 지침이 적용됨.
 - paths 프론트매터가 없는 경우 무조건 모든 파일에 적용됨

```
your-project/  
├── .claude/  
│   ├── CLAUDE.md           # Main project instructions  
│   └── rules/  
│       ├── code-style.md    # Code style guidelines  
│       ├── testing.md       # Testing conventions  
│       └── security.md      # Security requirements
```

```
---  
paths: src/api/**/*.ts  
---  
  
# API Development Rules  
  
- All API endpoints must include input validation  
- Use the standard error response format  
- Include OpenAPI documentation comments
```

9. 새로운 기능

❖ 자동 메모리

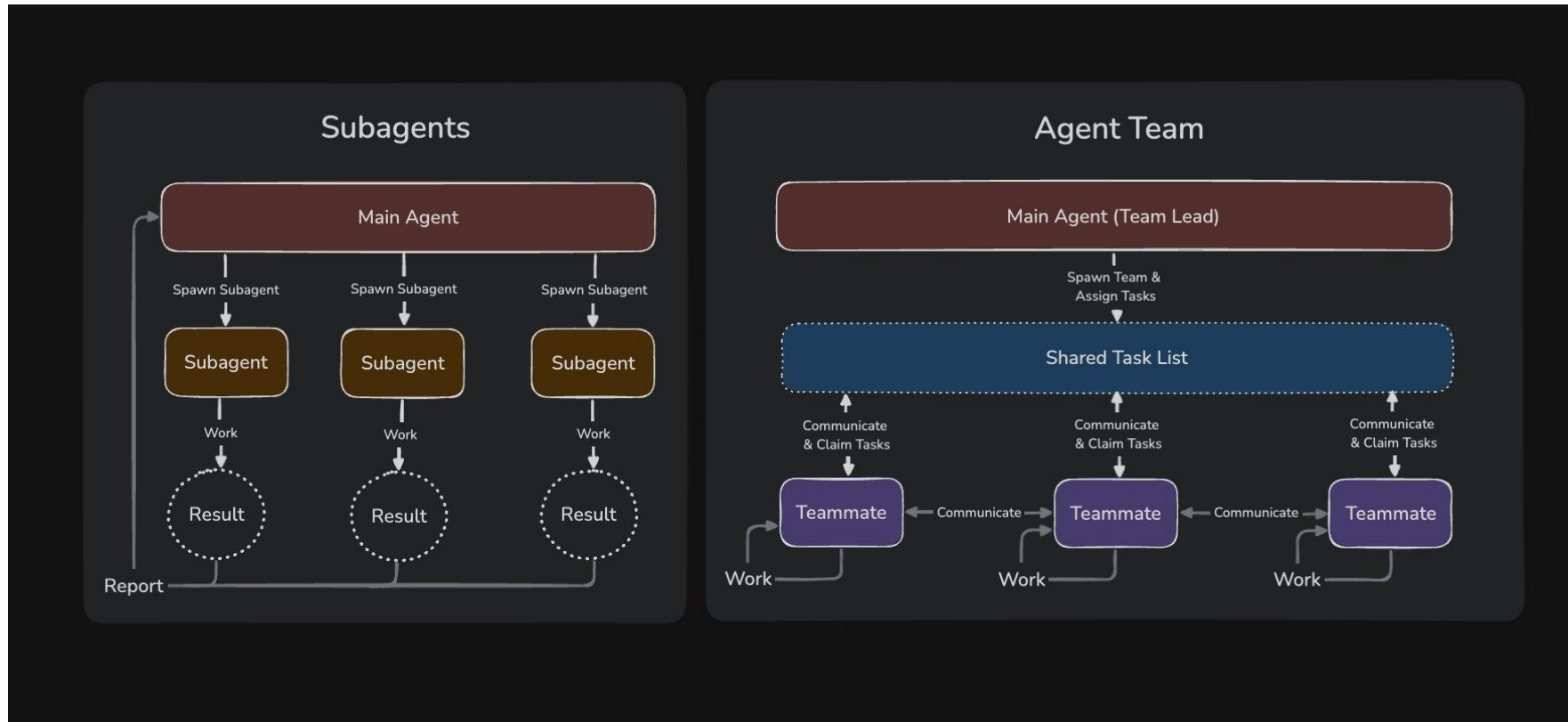
- Claude Code 2.1.59 이상에서 사용할 수 있음
- 기본적으로 사용가능하도록 설정되어 있음
 - /memory 명령어로 Auto-memory를 Off 시킬 수 있음
- Claude가 자동으로 사용자와의 대화 내용을 정리하고 학습함.
- 저장소 위치
 - ~/.claude/projects/프로젝트명/memory

```
~/.claude/projects/<project>/memory/  
├── MEMORY.md          # 간결한 인덱스, 모든 세션에 로드됨  
├── debugging.md       # 디버깅 패턴에 대한 자세한 노트  
├── api-conventions.md # API 설계 결정  
└── ...                # Claude가 만드는 다른 주제 파일
```

9. 새로운 기능

❖ Agent Team

- 전문 agent끼리 컨텍스트를 공유하며 협업하여 개발함
 - 공유 Task List 정보를 각 서브에이전트들이 공유함.
- 토큰 사용량이 과도함.



10. 기타 도구

❖ 다른 AI 코딩 도구가 필요한 이유

- Rate Limit : 유료 플랜을 사용하더라도 사용량이 많으면 사용제한이 발생함
- 다시 Rate Limit이 초기화될때까지 사용할 도구가 필요함
- 다음 무료 도구에 대한 사용법은 별도로 PDF 파일로 제공

❖ 대표적인 무료 도구

- antigravity-cli : Google
 - <https://antigravity.google/download>
 - Google Account를 이용해 로그인하면 하루당 1000번 요청이 Free Tier로 가능함
 - 5시간마다 할당량 초기화
 - 단 pro 모델 사용 불가, flash 모델만 사용할 수 있음
- codex : Open AI
 - <https://developers.openai.com/codex/cli/>
 - <https://github.com/openai/codex>
 - Chat GPT Plus Plan 이상이면 사용 가능
- OpenCode
 - 오픈소스 기반의 무료 AI 개발 도구
 - 일부 모델 무료이며, 주요 업체 AI 모델은 API Key, OAuth 2 인증 기반으로 연동 가능

11. 정리

❖ 도구 분류

- IDE형 (VSCode+AI 확장, Cursor/Windsurf) vs CLI형 (Claude Code, gemini-cli, Codex, qwen-code)로 구성.

❖ MCP 통합

- Settings에서 MCP 서버 등록 (예: context7, sequential-thinking, playwright, shadcn, supabase, github, vercel, k8s)로 외부 데이터·도구 안전 연동.

❖ Claude Code

- /init, /doctor, /review, /agents, /mcp 등 슬래시 명령어
- 권한 모드, system-prompt 설정
- SubAgent, Hook (Pre/PostToolUse, Stop 등)로 자동화·품질 제어
- 실무 팁
 - Plan Mode (승인 게이트), 계층적 CLAUDE.md
 - 정기적인 /compact로 컨텍스트 관리, 로컬 MCP 선호·보안 가드레일로 안정성 확보.